

Язык программирования C++

Лекция 20: Race condition, mutex, atomic, регулярные выражения, static_assert, compile-time вычисления, variadic templates

Никита Лисица

2026

- С точки зрения потока/процесса, переключение контекста может произойти в произвольный момент времени (в том числе в середине выполнения какой-нибудь операции)

- С точки зрения потока/процесса, переключение контекста может произойти в произвольный момент времени (в том числе в середине выполнения какой-нибудь операции)
- Как правило, мы не можем явно управлять распределением потоков по ядрам CPU, порядком их выполнения, и т.д.

Race condition

- С точки зрения потока/процесса, переключение контекста может произойти в произвольный момент времени (в том числе в середине выполнения какой-нибудь операции)
- Как правило, мы не можем явно управлять распределением потоков по ядрам CPU, порядком их выполнения, и т.д.
- Если результат выполнения программы зависит от порядка, в котором выполнялись инструкции в разных потоках, говорят о *состоянии гонки* (*race condition*)

Race condition: пример

```
#include <thread>
#include <iostream>

void hello() {
    std::cout << "Hello from " << std::this_thread::get_id()
        << std::endl;
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 5; ++i)
        threads.emplace_back(&hello);
    for (auto & thread : threads)
        thread.join();
}
```

Race condition: пример

```
#include <thread>
#include <iostream>

void hello() {
    std::cout << "Hello from " << std::this_thread::get_id()
               << std::endl;
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 5; ++i)
        threads.emplace_back(&hello);
    for (auto & thread : threads)
        thread.join();
}
```

- Что выведется на экран?

Race condition: пример

- Может быть

```
Hello from 140276650997504  
Hello from 140276667782912  
Hello from 140276659390208  
Hello from 140276642604800  
Hello from 140276676175616
```

Race condition: пример

- Может быть

```
Hello from 140276650997504
Hello from 140276667782912
Hello from 140276659390208
Hello from 140276642604800
Hello from 140276676175616
```

- А может быть

```
Hello from Hello from Hello from
140276650997504Hello
from 140276659390208Hello from
140276667782912
140276676175616
140276642604800
```

Race condition: пример

- Один поток добавляет элементы в связный список, второй поток итерируется по нему

Race condition: пример

- Один поток добавляет элементы в связный список, второй поток итерируется по нему

```
auto next = curr.next;  
curr.next = new_node;  
// Поток #1 остановился здесь  
new_node.next = next;
```

Race condition: пример

- Один поток добавляет элементы в связный список, второй поток итерируется по нему

```
auto next = curr.next;  
curr.next = new_node;  
// Поток #1 остановился здесь  
new_node.next = next;
```

- Во втором потоке сломается итерация по списку (например, если `new_node.next` было не проинициализировано)

Race condition: пример

```
int x = 0;

// Поток #1
while (!stopped) {
    ++x;
}

// Поток #2
while (!stopped) {
    if (x % 2 == 0) {
        std::cout << x;
    }
}
```

Race condition: пример

```
int x = 0;

// Поток #1
while (!stopped) {
    ++x;
}

// Поток #2
while (!stopped) {
    if (x % 2 == 0) {
        std::cout << x;
    }
}
```

- Может быть выведено нечётное число

Race condition: пример

```
int x = 0;

// Поток #1
while (!stopped) {
    ++x;
}

// Поток #2
while (!stopped) {
    if (x % 2 == 0) {
        std::cout << x;
    }
}
```

- Может быть выведено нечётное число
- Поток #2 выполнил проверку и зашёл внутрь `if`, затем передал управление потоку #1

Race condition: пример

```
int x = 0;

// Поток #1
while (!stopped) {
    ++x;
}

// Поток #2
while (!stopped) {
    if (x % 2 == 0) {
        std::cout << x;
    }
}
```

- Может быть выведено нечётное число
- Поток #2 выполнил проверку и зашёл внутрь **if**, затем передал управление потоку #1
- Поток #1 изменил **x** на нечётное значение и передал управление потоку #2

Гонка данных (data race)

- Когда один поток пишет в некоторую память, и одновременно один или несколько потоков эту память читают, говорят о *гонке данных* (*data race*)

Гонка данных (data race)

- Когда один поток пишет в некоторую память, и одновременно один или несколько потоков эту память читают, говорят о *гонке данных (data race)*
- В общем случае результат гонки данных – неопределённое поведение

Гонка данных (data race)

- Когда один поток пишет в некоторую память, и одновременно один или несколько потоков эту память читают, говорят о *гонке данных (data race)*
- В общем случае результат гонки данных – неопределённое поведение
- На некоторых архитектурах некоторые инструкции атомарны (напр. чтение или запись до 8 байт по выровненному адресу на x86-64)

Гонка данных (data race)

- Когда один поток пишет в некоторую память, и одновременно один или несколько потоков эту память читают, говорят о *гонке данных (data race)*
- В общем случае результат гонки данных – неопределённое поведение
- На некоторых архитектурах некоторые инструкции атомарны (напр. чтение или запись до 8 байт по выровненному адресу на x86-64)
- Как правило, операции в языке программирования компилируются в *несколько* инструкций CPU ($\implies$ не являются атомарными)

Гонка данных (data race)

- Когда один поток пишет в некоторую память, и одновременно один или несколько потоков эту память читают, говорят о *гонке данных (data race)*
- В общем случае результат гонки данных – неопределённое поведение
- На некоторых архитектурах некоторые инструкции атомарны (напр. чтение или запись до 8 байт по выровненному адресу на x86-64)
- Как правило, операции в языке программирования компилируются в *несколько* инструкций CPU ($\implies$ не являются атомарными)
- Оптимизации компилятора могут перемешивать инструкции, меняя поведение в многопоточном случае

Гонка данных: пример

```
int x = 0;

std::vector<std::thread> threads;
for (int i = 0; i < 5; ++i)
    threads.emplace_back([&x]{
        for (int i = 0; i < 1000; ++i)
            ++x;
    });

for (auto & thread : threads)
    thread.join();

std::cout << x << std::endl;
```

Гонка данных: пример

```
int x = 0;

std::vector<std::thread> threads;
for (int i = 0; i < 5; ++i)
    threads.emplace_back([&x]{
        for (int i = 0; i < 1000; ++i)
            ++x;
    });

for (auto & thread : threads)
    thread.join();

std::cout << x << std::endl;
```

- Что выведется на экран?

Гонка данных: пример

```
int x = 0;

std::vector<std::thread> threads;
for (int i = 0; i < 5; ++i)
    threads.emplace_back([&x]{
        for (int i = 0; i < 1000; ++i)
            ++x;
    });

for (auto & thread : threads)
    thread.join();

std::cout << x << std::endl;
```

- Что выведется на экран?
- В общем случае – что угодно (UB), на x86-64 – что угодно от 1000 до 5000

- `++x` это, на самом деле, три операции:

- `++x` это, на самом деле, три операции:
 - Прочитать `x` из памяти в регистр

- `++x` это, на самом деле, три операции:
 - Прочитать `x` из памяти в регистр
 - Инкрементировать регистр

- `++x` это, на самом деле, три операции:
 - Прочитать `x` из памяти в регистр
 - Инкрементировать регистр
 - Записать значение регистра обратно в память

- `++x` это, на самом деле, три операции:
 - Прочитать `x` из памяти в регистр
 - Инкрементировать регистр
 - Записать значение регистра обратно в память
- Возможное исполнение:

- `++x` это, на самом деле, три операции:
 - Прочитать `x` из памяти в регистр
 - Инкрементировать регистр
 - Записать значение регистра обратно в память
- Возможное исполнение:
 - Поток #1 прочитал `x`

- `++x` это, на самом деле, три операции:
 - Прочитать `x` из памяти в регистр
 - Инкрементировать регистр
 - Записать значение регистра обратно в память
- Возможное исполнение:
 - Поток #1 прочитал `x`
 - Поток #2 прочитал `x`

- `++x` это, на самом деле, три операции:
 - Прочитать `x` из памяти в регистр
 - Инкрементировать регистр
 - Записать значение регистра обратно в память
- Возможное исполнение:
 - Поток #1 прочитал `x`
 - Поток #2 прочитал `x`
 - Оба потока инкрементировали регистр

- `++x` это, на самом деле, три операции:
 - Прочитать `x` из памяти в регистр
 - Инкрементировать регистр
 - Записать значение регистра обратно в память
- Возможное исполнение:
 - Поток #1 прочитал `x`
 - Поток #2 прочитал `x`
 - Оба потока инкрементировали регистр
 - Поток #1 записал `x`

- `++x` это, на самом деле, три операции:
 - Прочитать `x` из памяти в регистр
 - Инкрементировать регистр
 - Записать значение регистра обратно в память
- Возможное исполнение:
 - Поток #1 прочитал `x`
 - Поток #2 прочитал `x`
 - Оба потока инкрементировали регистр
 - Поток #1 записал `x`
 - Поток #2 записал `x`

- $++x$ это, на самом деле, три операции:
 - Прочитать x из памяти в регистр
 - Инкрементировать регистр
 - Записать значение регистра обратно в память
- Возможное исполнение:
 - Поток #1 прочитал x
 - Поток #2 прочитал x
 - Оба потока инкрементировали регистр
 - Поток #1 записал x
 - Поток #2 записал x
 - $\implies x$ изменился с 0 на 1

- Мьютекс (*mutex*, от англ. *mutual exclusion*) – примитив синхронизации, ограничивающий доступ потоков к данным или коду

- Мьютекс (*mutex*, от англ. *mutual exclusion*) – примитив синхронизации, ограничивающий доступ потоков к данным или коду
- Часть кода, защищённая мьютексом (т.н. *критическая секция*) может одновременно выполняться **только одним потоком** (остальные будут **ждать** своей очереди)

- Мьютекс (*mutex*, от англ. *mutual exclusion*) – примитив синхронизации, ограничивающий доступ потоков к данным или коду
- Часть кода, защищённая мьютексом (т.н. *критическая секция*) может одновременно выполняться **только одним потоком** (остальные будут **ждать** своей очереди)
- В стандартной библиотеке есть реализация мьютекса `std::mutex`

```
std::mutex mutex;  
int x = 0;  
  
// ...  
  
threads.emplace_back([&x]{  
    for (int i = 0; i < 1000; ++i) {  
        // Код между lock и unlock - критическая секция  
        mutex.lock();  
        ++x;  
        mutex.unlock();  
    }  
});
```

- Почти всегда `mutex.lock()` и `mutex.unlock()` идут в паре

Мьютекс: `lock_guard`

- Почти всегда `mutex.lock()` и `mutex.unlock()` идут в паре
- Чтобы не забыть сделать `mutex.unlock()` (иначе потоки могут зависнуть!), есть `std::lock_guard`

```
// Сделает lock() в конструкторе и unlock() в деструкторе  
std::lock_guard<std::mutex> lock{mutex};  
++x;
```

Мьютекс: `lock_guard`

- Почти всегда `mutex.lock()` и `mutex.unlock()` идут в паре
- Чтобы не забыть сделать `mutex.unlock()` (иначе потоки могут зависнуть!), есть `std::lock_guard`

```
// Сделает lock() в конструкторе и unlock() в деструкторе  
std::lock_guard<std::mutex> lock{mutex};  
++x;
```

```
// Можно воспользоваться STAD  
std::lock_guard lock{mutex};  
++x;
```

- Можно отдать в `std::lock_guard` уже заблокированный мьютекс:

```
mutex.lock();  
// Сделает unlock() в деструкторе  
std::lock_guard lock{mutex, std::adopt_lock};  
++x;
```

- Если может потребоваться сбросить состояние заблокированности до деструктора, или переместить (move) владение блокировкой, есть `std::unique_lock`

- Если может потребоваться сбросить состояние заблокированности до деструктора, или переместить (move) владение блокировкой, есть `std::unique_lock`

```
std::unique_lock lock{mutex};  
++x;  
if (x > 10) {  
    // Можно вызвать unlock() самим  
    lock.unlock();  
    return;  
}  
++x;
```

Мьютекс: `unique_lock`

```
std::unique_lock lock{mutex, std::defer_lock};  
// После этого мьютекс будет заблокирован  
lock.lock();
```

Мьютекс: unique_lock

```
std::unique_lock lock{mutex, std::defer_lock};  
// После этого мьютекс будет заблокирован  
lock.lock();
```

```
std::unique_lock lock{mutex, std::try_lock};  
if (lock.owns_lock()) {  
    // Получилось заблокировать, можно войти  
    // в критическую секцию  
} else {  
    // Не получилось заблокировать - другой поток  
    // уже находится в критической секции  
}
```

Взаимная блокировка (deadlock)

```
// Поток #1  
mutex1.lock();  
mutex2.lock();
```

```
// Поток #2  
mutex2.lock();  
mutex1.lock();
```

Взаимная блокировка (deadlock)

```
// Поток #1  
mutex1.lock();  
mutex2.lock();
```

```
// Поток #2  
mutex2.lock();  
mutex1.lock();
```

- Поток #1 заблокировал `mutex1`, а поток #2 заблокировал `mutex2`

Взаимная блокировка (deadlock)

```
// Поток #1  
mutex1.lock();  
mutex2.lock();
```

```
// Поток #2  
mutex2.lock();  
mutex1.lock();
```

- Поток #1 заблокировал `mutex1`, а поток #2 заблокировал `mutex2`
- Теперь поток #1 ждёт разблокировки `mutex2`, а поток #2 ждёт разблокировки `mutex1`

Взаимная блокировка (deadlock)

```
// Поток #1  
mutex1.lock();  
mutex2.lock();
```

```
// Поток #2  
mutex2.lock();  
mutex1.lock();
```

- Поток #1 заблокировал `mutex1`, а поток #2 заблокировал `mutex2`
- Теперь поток #1 ждёт разблокировки `mutex2`, а поток #2 ждёт разблокировки `mutex1`
- $\implies$ мьютексы никогда не будут разблокированы, потоки зависнут

Взаимная блокировка (deadlock)

```
// Поток #1  
mutex1.lock();  
mutex2.lock();
```

```
// Поток #2  
mutex2.lock();  
mutex1.lock();
```

- Поток #1 заблокировал `mutex1`, а поток #2 заблокировал `mutex2`
- Теперь поток #1 ждёт разблокировки `mutex2`, а поток #2 ждёт разблокировки `mutex1`
- $\implies$ мьютексы никогда не будут разблокированы, потоки зависнут
- Эта ситуация называется *взаимной блокировкой (deadlock)*

Взаимная блокировка (deadlock)

- Существуют алгоритмы предотвращения взаимной блокировки (*deadlock avoidance*)

Взаимная блокировка (deadlock)

- Существуют алгоритмы предотвращения взаимной блокировки (*deadlock avoidance*)
- В стандартной библиотеке есть функция `std::lock(mutex1, mutex2, ...)`, реализующая такой алгоритм

Взаимная блокировка (deadlock)

- Существуют алгоритмы предотвращения взаимной блокировки (*deadlock avoidance*)
- В стандартной библиотеке есть функция `std::lock(mutex1, mutex2, ...)`, реализующая такой алгоритм
- Есть RAII-обёртка `std::scoped_lock`, делающая то же самое (и разблокировку в деструкторе)

Предотвращение взаимной блокировки

```
std::lock(mutex1, mutex2);  
std::lock_guard lock1(mutex1, std::adopt_lock);  
std::lock_guard lock2(mutex2, std::adopt_lock);  
// ...
```

Предотвращение взаимной блокировки

```
std::lock(mutex1, mutex2);  
std::lock_guard lock1(mutex1, std::adopt_lock);  
std::lock_guard lock2(mutex2, std::adopt_lock);  
// ...
```

```
std::unique_lock lock1(mutex1, std::defer_lock);  
std::unique_lock lock2(mutex2, std::defer_lock);  
std::lock(lock1, lock2);  
// ...
```

Предотвращение взаимной блокировки

```
std::lock(mutex1, mutex2);  
std::lock_guard lock1(mutex1, std::adopt_lock);  
std::lock_guard lock2(mutex2, std::adopt_lock);  
// ...
```

```
std::unique_lock lock1(mutex1, std::defer_lock);  
std::unique_lock lock2(mutex2, std::defer_lock);  
std::lock(lock1, lock2);  
// ...
```

```
std::scoped_lock lock(mutex1, mutex2);  
// ...
```

- Обычный мьютекс нельзя заблокировать дважды одним и тем же потоком (undefined behavior, в некоторых реализациях – `std::system_error`)

Рекурсивный мьютекс

- Обычный мьютекс нельзя заблокировать дважды одним и тем же потоком (undefined behavior, в некоторых реализациях – `std::system_error`)
- Для такого существует рекурсивный/reentrant мьютекс:

```
std::recursive_mutex mutex;  
  
void foo() {  
    std::lock_guard lock(mutex);  
    ...  
    if (...) {  
        foo();  
    }  
}
```

Атомарные типы (atomics)

- Атомарные операции – операции, которые выполняются целиком, не прерываясь и не перемешиваясь с другими операциями (в т.ч. других потоков)

Атомарные типы (atomics)

- Атомарные операции – операции, которые выполняются целиком, не прерываясь и не перемешиваясь с другими операциями (в т.ч. других потоков)
- Даже если архитектура CPU (напр. x86-64) гарантирует атомарность чтения/записи в каких-то случаях, компилятор может перемешать эти инструкции с другими, что повлияет на логику программы

Атомарные типы (atomics)

- Атомарные операции – операции, которые выполняются целиком, не прерываясь и не перемешиваясь с другими операциями (в т.ч. других потоков)
- Даже если архитектура CPU (напр. x86-64) гарантирует атомарность чтения/записи в каких-то случаях, компилятор может перемешать эти инструкции с другими, что повлияет на логику программы
- В стандартной библиотеке есть шаблон `std::atomic`, реализующий атомарные операции

Атомарные типы (atomics)

- Атомарные операции – операции, которые выполняются целиком, не прерываясь и не перемешиваясь с другими операциями (в т.ч. других потоков)
- Даже если архитектура CPU (напр. x86-64) гарантирует атомарность чтения/записи в каких-то случаях, компилятор может перемешать эти инструкции с другими, что повлияет на логику программы
- В стандартной библиотеке есть шаблон `std::atomic`, реализующий атомарные операции

```
std::atomic<int> count = 0;

// Из нескольких потоков
for (int i = 0; i < 1000; ++i)
    // Атомарно прибавить 1
    count.fetch_add(1);
```

Атомарные типы (atomics)

- В `std::atomic` можно завернуть любой примитивный тип, указатель, и простые структуры

Атомарные типы (atomics)

- В `std::atomic` можно завернуть любой примитивный тип, указатель, и простые структуры
- `load()` – атомарно прочитать значение (возвращает копию)

Атомарные типы (atomics)

- В `std::atomic` можно завернуть любой примитивный тип, указатель, и простые структуры
- `load()` – атомарно прочитать значение (возвращает копию)
- `store(value)` – атомарно записать значение

Атомарные типы (atomics)

- В `std::atomic` можно завернуть любой примитивный тип, указатель, и простые структуры
- `load()` – атомарно прочитать значение (возвращает копию)
- `store(value)` – атомарно записать значение
- `exchange(value)` – атомарно записать новое значение и вернуть старое

Атомарные типы (atomics)

- В `std::atomic` можно завернуть любой примитивный тип, указатель, и простые структуры
- `load()` – атомарно прочитать значение (возвращает копию)
- `store(value)` – атомарно записать значение
- `exchange(value)` – атомарно записать новое значение и вернуть старое
- `compare_exchange_weak/strong(value)` – реализация compare-and-swap (CAS)

Атомарные типы (atomics)

- В `std::atomic` можно завернуть любой примитивный тип, указатель, и простые структуры
- `load()` – атомарно прочитать значение (возвращает копию)
- `store(value)` – атомарно записать значение
- `exchange(value)` – атомарно записать новое значение и вернуть старое
- `compare_exchange_weak/strong(value)` – реализация compare-and-swap (CAS)
- Есть особые методы для числовых типов: `fetch_add(value)`, `fetch_sub(value)`, `fetch_min(value)`, `fetch_max(value)`, `fetch_and(value)`, `fetch_or(value)`, `fetch_xor(value)`

Атомарные типы (atomics)

- `std::atomic` может быть реализован через мьютекс, а может – через специфичные для платформы механизмы

Атомарные типы (atomics)

- `std::atomic` может быть реализован через мьютекс, а может – через специфичные для платформы механизмы
- Проверить, что `atomic` не использует мьютексы:
`is_lock_free()`, `is_always_lock_free`

Атомарные типы (atomics)

- `std::atomic` может быть реализован через мьютекс, а может – через специфичные для платформы механизмы
- Проверить, что `atomic` не использует мьютексы:
`is_lock_free()`, `is_always_lock_free`
- **NB:** Мьютексы можно реализовать через `atomic` используя CAS (т.н. *spinlock*)

- Мьютексы/атомики не только исполняют специфичные инструкции процессора, но и говорят компилятору о том, что определённые операции нельзя перемешивать

Мьютексы, atomics, и оптимизации

- Мьютексы/атомики не только исполняют специфичные инструкции процессора, но и говорят компилятору о том, что определённые операции нельзя перемешивать
- Почти все функции работы с атомиками принимают аргумент *memory order*, описывающий, какие именно гарантии нам нужны от этой операции (по умолчанию – *sequential consistency*, самая сильная гарантия)

Мьютексы, atomics, и оптимизации

- Мьютексы/атомики не только исполняют специфичные инструкции процессора, но и говорят компилятору о том, что определённые операции нельзя перемешивать
- Почти все функции работы с атомиками принимают аргумент *memory order*, описывающий, какие именно гарантии нам нужны от этой операции (по умолчанию – *sequential consistency*, самая сильная гарантия)
- Есть древний совет использовать **volatile** переменные в качестве атомиков – в современных CPU И компиляторах это не работает!

Регулярные выражения (regex)

- В стандартной библиотеке есть реализация регулярных выражений – класс `std::regex`

Регулярные выражения (regex)

- В стандартной библиотеке есть реализация регулярных выражений – класс `std::regex`
- Поддерживает много вариаций регулярных выражений (ECMAScript, basic POSIX, extended POSIX, ...)

Регулярные выражения (regex)

- В стандартной библиотеке есть реализация регулярных выражений – класс `std::regex`
- Поддерживает много вариаций регулярных выражений (ECMAScript, basic POSIX, extended POSIX, ...)

```
std::regex number_regex("\\d+");  
std::smatch match;  
std::regex_search("1234", match, number_regex);
```

Регулярные выражения (regex)

- В стандартной библиотеке есть реализация регулярных выражений – класс `std::regex`
- Поддерживает много вариаций регулярных выражений (ECMAScript, basic POSIX, extended POSIX, ...)

```
std::regex number_regex("\\d+");  
std::smatch match;  
std::regex_search("1234", match, number_regex);
```

- `std::regex_match` – проверка на соответствие регулярному выражению

Регулярные выражения (regex)

- В стандартной библиотеке есть реализация регулярных выражений – класс `std::regex`
- Поддерживает много вариаций регулярных выражений (ECMAScript, basic POSIX, extended POSIX, ...)

```
std::regex number_regex("\\d+");  
std::smatch match;  
std::regex_search("1234", match, number_regex);
```

- `std::regex_match` – проверка на соответствие регулярному выражению
- `std::regex_search` – поиск подстроки по регулярному выражению

Регулярные выражения (regex)

- В стандартной библиотеке есть реализация регулярных выражений – класс `std::regex`
- Поддерживает много вариаций регулярных выражений (ECMAScript, basic POSIX, extended POSIX, ...)

```
std::regex number_regex("(\\d)+");  
std::smatch match;  
std::regex_search("1234", match, number_regex);
```

- `std::regex_match` – проверка на соответствие регулярному выражению
- `std::regex_search` – поиск подстроки по регулярному выражению
- `std::regex_replace` – замена по регулярному выражению

- Позволяет сделать compile-time проверку некоторого условия:

```
// Текст ошибки опционален
static_assert(sizeof(int) == 4, "unsupported platform");

struct mesh_vertex {
    vec3f position;
    vec3f normal;
    vec2f texcoord;
};

static_assert(sizeof(mesh_vertex) == 32);

static_assert(std::atomic<int>::is_lock_free);
```

- Шаблоны в C++ можно рассматривать как функциональный язык программирования, выполняющийся в процессе компиляции (compile-time)

Compile-time вычисления

- Шаблоны в C++ можно рассматривать как функциональный язык программирования, выполняющийся в процессе компиляции (compile-time)

```
template <unsigned int N>
struct factorial {
    static const auto value = N * factorial<N - 1>::value;
};

template <>
struct factorial<0> {
    static const unsigned int value = 1;
};

auto x = factorial<5>::value; // x = 120
```

Compile-time вычисления

- Односвязный СПИСОК ТИПОВ:

```
struct empty{};

template <typename Head, typename Tail>
struct append {
    using head = Head;
    using tail = Tail;
};

template <typename List>
struct list_length;

template <>
struct list_length<empty> {
    static const size_t value = 0;
};

template <typename Head, typename Tail>
struct list_length<append<Head, Tail>> {
    static const size_t value = 1 + list_length<Tail>::value;
};
```

Compile-time вычисления: constexpr

- Ключевое слово **constexpr** означает, что переменная или функция *могут* быть вычислены в compile-time

Compile-time вычисления: constexpr

- Ключевое слово **constexpr** означает, что переменная или функция *могут* быть вычислены в compile-time

```
constexpr unsigned int factorial(unsigned int n) {  
    if (n == 0)  
        return 1;  
    return n * factorial(n - 1);  
}
```

Compile-time вычисления: constexpr

- Ключевое слово **constexpr** означает, что переменная или функция *могут* быть вычислены в compile-time

```
constexpr unsigned int factorial(unsigned int n) {  
    if (n == 0)  
        return 1;  
    return n * factorial(n - 1);  
}
```

- На такую функцию есть ограничения (не может быть виртуальной, не может иметь побочные эффекты, и т.п.)

- `constexpr` переменная будет автоматически `const`

- `constexpr` переменная будет автоматически `const`
- `consteval` функция **всегда** вычисляется в compile-time

const, constexpr, consteval, constexpr

- `constexpr` переменная будет автоматически `const`
- `consteval` функция **всегда** вычисляется в compile-time
- `constexpr` переменная инициализируется в compile-time (но не будет автоматически `const`)

- Шаблон (функции или класса), параметризуемый произвольным списком типов (произвольной длины)

Variadic templates

- Шаблон (функции или класса), параметризуемый произвольным списком типов (произвольной длины)
- Синтаксис: `<typename ... Args>`

Variadic templates

- Шаблон (функции или класса), параметризующийся произвольным списком типов (произвольной длины)
- Синтаксис: `<typename ... Args>`

```
template <typename ... Args>  
void print(Args ... args);
```

Variadic templates

- Шаблон (функции или класса), параметризующийся произвольным списком типов (произвольной длины)
- Синтаксис: `<typename ... Args>`

```
template <typename ... Args>  
void print(Args ... args);
```

- К типам можно добавить уточнения (ссылки, `const`):

```
template <typename ... Args>  
void print(const Args& ... args);
```

- Сам набор аргументов **args** называется *variadic pack*

- Сам набор аргументов **args** называется *variadic pack*
- Выражение **args...** позволяет ‘раскрыть’ *variadic pack* и передать куда-нибудь (напр. в функцию)

Variadic templates

- Сам набор аргументов `args` называется *variadic pack*
- Выражение `args...` позволяет ‘раскрыть’ variadic pack и передать куда-нибудь (напр. в функцию)
- `sizeof...(Args)` – количество элементов в наборе

- Два способа использовать variadic pack: рекурсия и fold expressions

Variadic templates

- Два способа использовать variadic pack: рекурсия и fold expressions
- Пример рекурсии:

```
// Типичный подход: первый аргумент отдельно,  
// рекурсия по оставшимся аргументам  
template <typename T, typename ... Args>  
void print(const T& first, const Args& ... args) {  
    std::cout << first;  
    print(args...);  
}  
  
// База рекурсии  
void print() {}
```

- Правая свёртка: выражение $(\text{args} + \dots)$ превращается в $\text{arg0} + (\text{arg1} + (\text{arg2} + \dots))$

Fold expressions

- Правая свёртка: выражение $(\text{args} + \dots)$ превращается в $\text{arg0} + (\text{arg1} + (\text{arg2} + \dots))$
- Левая свёртка: выражение $(\dots + \text{args})$ превращается в $((\text{arg0} + \text{arg1}) + \text{arg2}) + \dots$

Fold expressions

- Правая свёртка: выражение $(args + \dots)$ превращается в $arg0 + (arg1 + (arg2 + \dots))$
- Левая свёртка: выражение $(\dots + args)$ превращается в $((arg0 + arg1) + arg2) + \dots$
- Правая свёртка с начальным значением: выражение $(args + \dots + init)$ превращается в $arg0 + (arg1 + (arg2 + \dots + (argn + init)))$

Fold expressions

- Правая свёртка: выражение $(args + \dots)$ превращается в $arg0 + (arg1 + (arg2 + \dots))$
- Левая свёртка: выражение $(\dots + args)$ превращается в $((arg0 + arg1) + arg2) + \dots$
- Правая свёртка с начальным значением: выражение $(args + \dots + init)$ превращается в $arg0 + (arg1 + (arg2 + \dots + (argn + init)))$
- Левая свёртка с начальным значением: выражение $(init \dots + args)$ превращается в $((init + arg0) + arg1) + arg2) + \dots$

Fold expressions

```
template <typename ... Args>  
auto sum(const Args& ... args) {  
    return (args + ...);  
}
```

Fold expressions

```
template <typename ... Args>  
auto sum(const Args& ... args) {  
    return (args + ...);  
}
```

```
template <typename ... Args>  
void print(const Args& ... args) {  
    (std::cout << ... << args);  
}
```

Fold expressions

- Свёртку можно использовать почти со всеми операторами, включая оператор запятая – это позволяет реализовать ‘цикл по variadic pack’:

```
template <typename ... Args>
void print(const Args& ... args) {
    (std::cout << args, ...);
}
```

```
template <typename ... Args>
void clear_all(const Args& ... args) {
    (args.clear(), ...);
}
```